

Iterable - Iterator

Juan Ignacio Raggio

May 3, 2025

Contents

1	Iterator interface	1
2	Iterable interface	2
2.1	Notemos que lo que hace iterable es encapsular la interfaz iterator, tiene sentido porque en caso de que no usemos esta interfaz, tendríamos que crear una clase que solo tenga el iterador	2
3	Trampas en ejercicios de iterators	2

1 Iterator interface

```
public interface Iterator<E> {  
    boolean hasNext();  
  
    E next();  
  
    default void remove() {  
        throw new UnsupportedOperationException("remove");  
    }  
  
    default void forEachRemaining(Consumer<? super E> action) {  
        Objects.requireNonNull(action);  
        while (hasNext())  
            action.accept(next());  
    }  
}
```

2 Iterable interface

- 2.1 Notemos que lo que hace iterable es encapsular la interfaz iterator, tiene sentido porque en caso de que no usemos esta interfaz, tendríamos que crear una clase que solo tenga el iterador

```
public interface Iterable<T> {
    Iterator<T> iterator();

    default void forEach(Consumer<? super T> action) {
        Objects.requireNonNull(action);
        // Notemos que podemos usar un foreach porque ya sabemos que this es iterable
        for (T t : this) {
            action.accept(t);
        }
    }

    default Spliterator<T> spliterator() {
        return Spliterators.spliteratorUnknownSize(iterator(), 0);
    }
}
```

3 Trampas en ejercicios de iterators

- Pueden pedirme que implemente un iterador y el mismo tiene que ser un iterador en si
- Si en el programa de prueba no me muestran una invocacion al metodo `iterator()` y se lo guardan en una variable, quiere decir que el constructor de ese iterador en si tiene que inicializar todo lo necesario para el iterador
- De hecho nisiquiera deberia implementar a iterable, a menos de que se meta en un `for-each` en tal caso si
- Ejemplo:

```
@Test
public void mainTest() {
    String[] v1 = new String[] { "hola", "mundo", "adios" };
}
```

```
String[] v2 = new String[] { "hello", "world" };

ParallelIterator<String> myIterator = new ParallelIterator<>(v1, v2);
System.out.println(myIterator.next());
System.out.println(myIterator.next());
}
```

- Notemos que a partir de myIterator se llama a next entonces lo que tendria que pasar en parallelIterator es lo siguiente:

```
public class ParallelIterator<T> {

    private int ACurrent, BCurrent;
    private T[] A, B;

    private void verifyNonNullCollection(Object param, String description) {
        if (param == null)
            throw new RuntimeException("%s collection missing".formatted(description));
    }

    public ParallelIterator(T[] collectionA, T[] collectionB) {
        verifyNonNullCollection(A, "First");
        verifyNonNullCollection(B, "Second");
        A = collectionA;
        B = collectionB;
        ACurrent = BCurrent = 0;
    }

    public boolean hasNext() {
        return ACurrent != A.length && BCurrent != B.length;
    }

    public Pair<T> next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        return new Pair<>(A[ACurrent++], B[BCurrent++]);
    }
}
```